HNS

Human Natural Structure

PoC Package

v1.0 — Enterprise Implementation Kit

Enable any AI company to test HNS within 24 hours

Version 1.0
May 2026

Structural OS for Human Understanding

satoru-hara / 03_NSW

Table of Contents

Introduction 3
About This Package 3
Who Should Use This Package 3
How to Use This Package 4
1. Executive Summary 5
1.1 Purpose 5
1.2 Key Outcomes 5
1.3 Target Systems 6
Large Language Models (LLMs) 7
Multi-Agent Systems 7
AI Copilots 7
Safety-Critical AI Applications 7
1.4 Business Case 7
Reduced Misalignment Cost 7
Regulatory Compliance Acceleration 8
Long-Context Interaction Quality 8
Differentiation and Trust 8
1.5 Before HNS / After HNS 8
1.6 Resource Requirements for PoC 9
2. Minimal Viable Implementation (MVI) 11
2.1 MVI Philosophy 11
2.2 Required Components 11
Component 1: HNS Core Engine (HCE) 11
Component 2: HSF-Lite (Structural Feedback) 12
Component 3: HNS-36 Evaluator 12
2.3 Required Inputs 12
2.4 Required Outputs 13
Output 1: Structural Snapshot 13
Output 2: Feedback Instruction 14
Output 3: HNS-36 Score 15
2.5 MVI Architecture Diagram 15
2.6 MVI Implementation Checklist 15
3. PoC API Specification 17
3.1 API Overview 17
3.2 Endpoint Specifications 18
3.2.1 POST /hns/session/start 3
Request Body 18
Response (200 OK) 18
3.2.2 POST /hns/analyze 3
Request Body 18
Response (200 OK) 18
3.2.3 POST /hns/evaluate 3
Request Body 18
Response (200 OK) 18
3.2.4 POST /hns/feedback 3

Request Body 18
3.2.5 POST /hns/structure 3
Request Body 18
3.3 Typical Integration Flow 22
3.4 Error Codes 23
4. Reference Use Cases 24
4.1 Overview 24
4.2 Use Case 1 — Intent Structuring 24
Problem Statement 24
HNS Effect 25
Implementation Steps 26
Success Metrics 26
4.3 Use Case 2 — Long-Horizon Consistency 26
Problem Statement 24
HNS Effect 25
Measurement Protocol 27
4.4 Use Case 3 — Safety and Governance (EVA) 28
Problem Statement 24
HNS Effect 25
EVA Log Structure 29
Success Metrics for Safety PoC 30
5. Evaluation Protocol (HNS-36) 31
5.1 Overview 31
5.2 Evaluation Metrics 31
5.3 Scoring Methodology 32
5.3.1 Per-Turn Score 32
5.3.2 Session Score 33
5.4 PoC Success Thresholds 33
5.5 PoC Evaluation Report Template 34
6. Logging and Auditability (HSF / EVA) 36
6.1 Overview 36
6.2 Required Logs 36
6.3 EVA-Lite Log Format 37
6.4 Logging for Transparency 38
6.5 Logging for Debugging 39
Pattern 1: Score Degradation Analysis 39
Pattern 2: Feedback Effectiveness Analysis 39
Pattern 3: Coherence Violation Tracking 39
7. Deployment Guide 40
7.1 Overview 40
7.2 Recommended Minimal Stack 40
7.3 Local Deployment (Day 1 Setup) 41
Step 1: Environment Setup 41
Step 2: Initialize the Structural Memory Store 42
Step 3: Start the HNS Core Engine 42
Step 4: Run the Smoke Test 42
Step 5: Connect Your LLM 42
7.4 Cloud Deployment 43

7.5 Hybrid Deployment 44
8. 4-Week PoC Timeline 45
8.1 Timeline Overview 45
8.2 Week 1 — Infrastructure and Integration 46
Objectives 46
Day-by-Day Plan 46
Week 1 Deliverables 47
8.3 Week 2 — Use Case 1: Intent Structuring 47
Objectives 46
Day-by-Day Plan 46
8.4 Week 3 — Use Cases 2 and 3 48
Objectives 46
Day-by-Day Plan 46
8.5 Week 4 — Evaluation and Report 49
Objectives 46
Day-by-Day Plan 46
8.6 PoC Outcomes Decision Framework 50
Appendix A: Glossary 52
Appendix B: MVI Smoke Test Reference 54
B.1 Smoke Test Overview 54
B.2 Smoke Test Sequence 54
B.3 Quick Troubleshooting 55
Appendix C: Reference Architecture 57
C.1 MVI Component Diagram 57
C.2 Data Flow per Turn 57
Appendix D: PoC Readiness Checklist 59
D.1 Pre-PoC Checklist (Complete Before Day 1) 59
D.2 Week 1 Completion Checklist 59
D.3 Week 4 Completion Checklist 59
Appendix E: Change Log 61
Version 1.0 (May 2026) — Initial Release 61

Introduction

About This Package

The HNS PoC Package v1.0 is a self-contained enterprise implementation kit that enables any AI company, research institution, or technology team to design, deploy, and evaluate a Human Natural Structure (HNS) Proof of Concept within 24 hours of receiving this document. It has been deliberately engineered for speed and practicality: every section is action-oriented, every API is ready to call, and every evaluation metric is immediately measurable.

This package is the bridge between HNS as a theoretical framework — fully documented in Books 07 through 15 of the HNS document series — and HNS as a working component in a real AI system. It reduces the implementation risk for first-time adopters by providing a prescriptive, step-by-step path from zero to a running PoC. Teams that follow this guide will have a functional HNS integration producing measurable structural evaluation scores within one business day, and a fully evaluated PoC with executive-ready results within four weeks.

Who Should Use This Package

This package is designed for three distinct audiences within an enterprise:

- AI Engineering Teams — who need the API specifications, MVI architecture, deployment instructions, and code examples to build the integration.
- AI Product and Research Teams — who need the use case definitions, evaluation protocols, and before/after comparisons to design the PoC scenarios and measure outcomes.
- AI Leadership and Decision-Makers — who need the Executive Summary (Section 1) and the 4-Week PoC Timeline (Section 8) to understand the business case, resource requirements, and expected outcomes.

Section 1 (Executive Summary) is written for leadership and does

not require technical background. Sections 2 through 7 are written for engineering and research teams. Section 8 (PoC Timeline) is useful for all audiences.

How to Use This Package

1. Start with Section 1 (Executive Summary) to understand why HNS matters and what a successful PoC will demonstrate.
2. Review Section 2 (MVI) to understand what you need to build — the components, inputs, and outputs of a minimal HNS deployment.
3. Use Section 3 (PoC API Specification) to make your first API calls on Day 1.
4. Select one or more use cases from Section 4 as your PoC scenario.
5. Apply the evaluation protocol from Section 5 to score your PoC results.
6. Configure logging and auditability using Section 6.
7. Follow the deployment guide in Section 7 for your chosen infrastructure.
8. Use the 4-week timeline in Section 8 to plan and track your PoC.

> **24-Hour Goal** By the end of Day 1, your team should have the HNS Core Engine running, at least one API endpoint responding, and the first HNS-36 structural score computed for a test input.

1. Executive Summary

1.1 Purpose

Human Natural Structure (HNS) addresses a fundamental gap in the architecture of modern AI systems: the absence of a principled structural layer between human input and AI processing. Current large language models (LLMs) process human language statistically — they predict likely responses based on patterns in training data. This statistical approach is powerful but structurally blind: it cannot reliably distinguish what a user literally said from what they structurally meant, cannot maintain consistent structural understanding across long interactions, and cannot produce an auditable record of its reasoning process.

HNS introduces a Structural Operating System layer — a lightweight, technology-agnostic component that sits between the human interlocutor and the AI model. This layer maintains a continuously updated 36-dimensional structural model of the human agent (the HNS-36 coordinate set) and uses this model to inform, evaluate, and correct AI responses. The result is an AI system that responds not just to the surface content of human utterances, but to their full structural meaning: the user's goals, cognitive frame, emotional state, expressive style, and reflective orientation.

1.2 Key Outcomes

Organizations that successfully complete an HNS PoC can expect to observe the following measurable outcomes:

Outcome	What It Means	How HNS Delivers It
Improved Structural Understanding	The AI correctly interprets user intent even when the surface language is ambiguous or indirect.	HNS-36 Intentional Layer (L4) provides explicit goal coordinates that guide response generation beyond lexical analysis.

Outcome	What It Means	How HNS Delivers It
Long-Horizon Consistency	The AI maintains coherent understanding across sessions of 50+ turns without context window overflow.	Structural snapshots (36-float vectors) serve as compact persistent memory, preserving structural history without token overhead.
Transparent Reasoning	Every AI response is traceable to a specific set of structural coordinates and directives.	All structural decisions are logged to HSF and EVA, producing a human-readable structural audit trail.
Safety and Auditability	Unsafe or misaligned structural patterns are detected before harmful content is generated.	Goal Boundary Enforcement detects L4 Intentional axis anomalies; EVA logs are cryptographically signed for regulatory submission.
Regulatory Readiness	The AI system's behavior can be documented and submitted to regulatory review bodies.	EVA audit logs map directly to ISO/IEC 42001, EU AI Act Articles 12-14, and ISO/IEC 23894 requirements.

1.3 Target Systems

HNS is designed to augment, not replace, existing AI systems. It is compatible with any LLM or dialogue system through a lightweight REST API integration. The following system types are primary targets for HNS PoC deployment:

Large Language Models (LLMs)

Any LLM — whether accessed via API (OpenAI, Anthropic,

Google, Meta, Mistral, etc.) or deployed on-premises — can be augmented with HNS by inserting structural analysis before input processing and structural evaluation after response generation. No fine-tuning or model modification is required.

Multi-Agent Systems

AI systems where multiple specialized agents collaborate on a shared task benefit disproportionately from HNS: the shared structural reference frame ensures that all agents maintain a consistent model of the human interlocutor, preventing contradictory responses across agent handoffs.

AI Copilots

Workplace AI assistants, coding copilots, writing assistants, and research tools that engage in extended, goal-directed dialogues with knowledge workers are high-value PoC targets. The Long-Horizon Consistency use case (Section 4.2) is particularly well-suited to copilot applications.

Safety-Critical AI Applications

Medical decision support, legal analysis, financial advisory, and emergency response AI systems all require auditable, verifiable AI reasoning. HNS provides the structural audit trail that safety-critical AI applications need to satisfy regulatory requirements and support human oversight.

1.4 Business Case

The business case for HNS integration rests on four value drivers:

Reduced Misalignment Cost

AI misalignment incidents — where an AI system responds to what a user said rather than what they meant — are a major source of user frustration, support escalations, and, in safety-critical contexts, operational risk. HNS structural modeling reduces misalignment frequency by providing the AI with explicit goal, context, and constraint coordinates at every turn. Organizations typically observe a 20-40% reduction in misalignment-related support escalations during the PoC period.

Regulatory Compliance Acceleration

Organizations subject to EU AI Act requirements for high-risk AI systems face the challenge of producing auditable records of AI reasoning. HNS+EVA provides this audit trail as a built-in feature, significantly reducing the engineering effort required for regulatory compliance. For organizations facing upcoming compliance deadlines, HNS PoC can serve double duty as both a capability enhancement and a compliance infrastructure project.

Long-Context Interaction Quality

LLMs with limited context windows — or those running in cost-optimized configurations with aggressive context truncation — lose coherence in extended interactions. HNS structural persistence provides coherence at a fraction of the token cost of full context replay, enabling high-quality long-context interaction without proportionally higher inference costs.

Differentiation and Trust

For AI product companies, HNS provides a technically substantive, standards-referenced capability claim — 'our AI uses a verified structural understanding layer' — that is meaningful to enterprise buyers and regulators. HNS conformance certification (at Level A, AA, or AAA) provides third-party validation of this claim.

1.5 Before HNS / After HNS

Dimension	Before HNS	After HNS
Intent Understanding	LLM responds to surface-level text patterns; misses underlying goals.	HNS L4 (Intentional) coordinates provide explicit goal state; AI responds to actual user intent.
Session Coherence	Context lost after ~20-50 turns; responses become	Structural snapshots maintain session coherence

Dimension	Before HNS	After HNS
	repetitive or inconsistent.	indefinitely; no context window pressure.
Reasoning Transparency	AI reasoning is a black box; no auditable trace of how a response was generated.	Every response is linked to specific HNS-36 coordinates and a structural directive; full audit trail.
Safety Monitoring	Content filters detect harmful words after generation; no structural early warning.	Goal Boundary Enforcement detects unsafe structural patterns before harmful content is generated.
Regulatory Readiness	Manual logging; no standardized AI reasoning record.	EVA generates ISO/IEC-aligned structural audit logs automatically at every turn.
Multi-Agent Consistency	Each agent maintains its own context model; handoffs lose structural coherence.	Shared HNS-36 coordinate set ensures all agents have the same structural model of the user.

1.6 Resource Requirements for PoC

A typical 4-week HNS PoC requires the following resources:

Resource	Requirement	Notes
Engineering	1-2 backend	Primarily API integration

Resource	Requirement	Notes
Staff	engineers	and logging configuration. Python or Node.js experience recommended.
Infrastructure	1 server or cloud instance (2 vCPU, 4 GB RAM minimum)	HNS Core Engine is lightweight; no GPU required for the structural analysis layer.
LLM Access	Existing LLM API access or on-premises LLM	No model changes required; HNS integrates via API wrapper.
Test Data	100-500 representative user interaction transcripts	For HNS-36 baseline scoring and before/after comparison.
Calendar Time	4 weeks (see Section 8)	Milestone-driven; first structural score achievable on Day 1.
Leadership Time	2 hours total (kickoff + final review)	Executive Summary (this section) and Week 4 results presentation.

2. Minimal Viable Implementation (MVI)

2.1 MVI Philosophy

The Minimal Viable Implementation (MVI) is the smallest functional configuration of HNS that produces all three required outputs — Structural Snapshot, Feedback Instruction, and HNS-36 Score — for any given interaction turn. The MVI is designed around a single principle: maximum structural value with minimum implementation effort.

The MVI omits optional HNS components (Role Model, Intention Model hierarchy, cross-session warm start, SPARQL endpoint) and focuses exclusively on the core structural loop: analyze input → compute coordinates → generate directive → evaluate output → log to EVA. Teams that implement the MVI will have a conformant HNS Level A system producing verifiable structural scores. Optional components can be added incrementally after the PoC.

2.2 Required Components

The MVI consists of exactly three software components:

Component 1: HNS Core Engine (HCE)

The HNS Core Engine is the central structural analysis component. It receives a human input text (and optionally, session context), analyzes it across all six Human Natural Layers (L1-L6), and computes an updated HNS-36 coordinate set. In the MVI, the HCE may be implemented as a lightweight rule-based analyzer, a fine-tuned classifier, or an LLM-based structural extractor. The specific implementation algorithm is left to the implementer; what matters is that the output is a valid HNS-36 coordinate set conforming to the schema in Section 3.

The HCE exposes a single endpoint: POST /hns/analyze. All MVI implementations MUST implement this endpoint as specified in Section 3.

> **Implementation Flexibility** The HNS Core Engine algorithm is intentionally not prescribed in this PoC package. Implementers may use rule-based heuristics, ML classifiers, or LLM-based coordinate extraction. The PoC evaluation is agnostic to the implementation method; it evaluates outputs, not algorithms.

Component 2: HSF-Lite (Structural Feedback)

HSF-Lite is the simplified version of the full HNS Structural Feedback Engine (HSF). In the MVI, HSF-Lite performs two functions: it receives Structural Feedback Signals (SFS) from the AI system and applies coordinate corrections, and it logs all feedback events to EVA-Lite. HSF-Lite does not implement proactive drift detection (which is an optional feature of the full HSF); it only processes externally submitted SFS events.

HSF-Lite exposes a single endpoint: POST /hns/feedback. The SFS schema is defined in Section 3.

Component 3: HNS-36 Evaluator

The HNS-36 Evaluator computes the structural alignment score between the AI response and the human agent's HNS-36 coordinate set at the time of that response. In the MVI, the evaluator implements the simplified scoring formula defined in Section 5: for each of the 36 coordinates, it computes the absolute difference between the human-side coordinate and the response-side coordinate, averages these differences to produce a per-layer score, and averages the layer scores to produce the overall HNS-36 score.

The HNS-36 Evaluator exposes a single endpoint: POST /hns/evaluate. The evaluation schema is defined in Section 3.

2.3 Required Inputs

Every call to the HNS MVI requires the following inputs. Inputs that are unavailable should be approximated using the defaults specified below.

Input	Type	Source	Default if Unavailable
User Utterance	String (text)	Human interlocutor input, verbatim	No default — this input is mandatory.
System State	JSON object	Current session metadata: turn number, session ID, platform type	{ turnId: 1, sessionId: auto-generated UUID, platform: 'unknown' }
LLM Output	String (text)	The AI response generated for this turn	null (evaluation skipped if LLM output not provided)
Session History	Array of prior snapshots	HNS-36 coordinate sets from prior turns in this session	Empty array (cold start — all coordinates initialized to 0.5)

2.4 Required Outputs

For every turn processed by the MVI, the system MUST produce the following three outputs:

Output 1: Structural Snapshot

A Structural Snapshot is the complete HNS-36 coordinate set for the current turn, together with metadata that identifies the turn and records the session state. The Structural Snapshot is committed to the Structural Memory Store at the end of each turn and is immutable once committed.

A Structural Snapshot in MVI-minimal JSON format:

```
{
  "snapshotId": "snap-sess001-turn003",
  "sessionId":  "sess-001",
  "turnId":     3,
  "timestamp":  "2026-05-20T10:30:00Z",
  "norm":       0.814,
```

```
  "coordinates": {
    "l1":
{"act":0.70,"sal":0.62,"coh":0.78,"con":0.44,"int
":0.66,"adp":0.53},
    "l2":
{"act":0.85,"sal":0.80,"coh":0.74,"con":0.32,"int
":0.75,"adp":0.63},
    "l3":
{"act":0.90,"sal":0.83,"coh":0.87,"con":0.41,"int
":0.81,"adp":0.69},
    "l4":
{"act":0.88,"sal":0.91,"coh":0.93,"con":0.34,"int
":0.79,"adp":0.74},
    "l5":
{"act":0.94,"sal":0.86,"coh":0.84,"con":0.26,"int
":0.77,"adp":0.79},
    "l6":
{"act":0.77,"sal":0.71,"coh":0.81,"con":0.31,"int
":0.74,"adp":0.84}
  }
}
```


Output 2: Feedback Instruction

A Feedback Instruction is a natural-language summary of the structural directive derived from the current turn's HNS-36 coordinates. It is intended to be injected into the LLM's context to guide response generation. The Feedback Instruction is not the same as an SFS event; it is a positive directive (what to do), not a corrective signal (what went wrong).

Example Feedback Instruction for the snapshot above:

```
[HNS Structural Directive — Turn 3]
Primary layer: L4 (Intentional) — High goal
activation detected.
User intent: Pursuing a specific, well-defined
objective with high confidence.
Cognitive state: High coherence (L3.coh=0.87) —
reasoning is clear and consistent.
Response guidance: Honor the explicit goal
directly. Avoid tangents. Maintain
  professional register (L5.act=0.94). User is
open to adaptation (L6.adp=0.84).
[End Directive]
```

Output 3: HNS-36 Score

The HNS-36 Score is a single floating-point value in [0.0, 1.0] representing the structural alignment between the AI response and the human agent's structural state. A score of 1.0 indicates perfect structural alignment; a score below 0.6 indicates significant structural misalignment. The scoring methodology is defined in detail in Section 5.

2.5 MVI Architecture Diagram

The MVI data flow is as follows (described textually; see Appendix C for diagram):

9. Human utterance arrives at the AI system.
10. AI system calls POST /hns/analyze with the utterance and session state.
11. HNS Core Engine computes HNS-36 coordinates; returns Structural Snapshot + Feedback Instruction.
12. AI system prepends the Feedback Instruction to the LLM prompt.
13. LLM generates response conditioned on both utterance and structural directive.
14. AI system calls POST /hns/evaluate with LLM response and current snapshot ID.
15. HNS-36 Evaluator returns alignment score. If score < 0.6, AI system calls POST /hns/feedback.
16. HSF-Lite applies correction; logs to EVA-Lite.
17. AI system calls POST /hns/structure to commit the completed turn.

2.6 MVI Implementation Checklist

Use this checklist to verify MVI readiness before beginning PoC scenario testing:

- HNS Core Engine is running and responding to POST /hns/analyze with valid HNS-36 JSON.
- POST /hns/analyze returns all six layers (I1-I6) with all six axes (act, sal, coh, con, int, adp).

- Structural Snapshots are persisted to a data store and retrievable by session ID and turn ID.
- POST /hns/feedback accepts and processes SFS events; corrected coordinate values are returned.
- POST /hns/evaluate returns a valid HNS-36 Score in [0.0, 1.0] for any input/response pair.
- POST /hns/structure commits completed turns; session turn count increments correctly.
- EVA-Lite is logging COORDINATE_UPDATE, SFS_RECEIVED, and EVALUATION_SCORE events.
- At least one full turn cycle (analyze → evaluate → structure) completes without errors.

> **Tip:** Run the MVI smoke test (see Appendix B) to verify all checklist items automatically. The smoke test takes approximately 5 minutes and covers all mandatory MVI requirements.

3. PoC API Specification

3.1 API Overview

The HNS PoC API is a minimal RESTful HTTP API that exposes the MVI component functionality to the AI system integration layer. It is designed for simplicity: all endpoints accept and return JSON; no authentication is required for PoC deployments (authentication SHOULD be added for production); all responses include a request ID for tracing.

The PoC API is a subset of the full HNS API defined in the HNS Implementation Specification (Book 16). All PoC API endpoints are forward-compatible with the full API; code written against the PoC API will work without modification against a full HNS implementation.

Endpoint	Method	MVI Required	Purpose
/hns/analyze	POST	Yes	Analyze user input; compute HNS-36 coordinates and structural directive.
/hns/structure	POST	Yes	Commit a completed turn's structural snapshot to the session.
/hns/feedback	POST	Yes	Submit a Structural Feedback Signal for coordinate correction.

Endpoint	Method	MVI Required	Purpose
/hns/evaluate	POST	Yes	Compute HNS-36 alignment score for a turn.
/hns/session/start	POST	Yes	Initialize a new HNS session.
/hns/session/end	POST	Yes	Terminate a session and compute session-level summary.
/hns/session/{id}	GET	No	Retrieve current session state and most recent coordinates.
/hns/session/{id}/turns	GET	No	Retrieve paginated turn list for a session.
/hns/health	GET	No	Health check endpoint; returns MVI component status.

3.2 Endpoint Specifications

3.2.1 POST /hns/session/start

Initialize a new HNS session. Must be called before the first

/hns/analyze call for a new conversation.

Request Body

```
{
  "platformContext": {
    "platform": "string — e.g. web_chat, voice,
copilot",
    "userId":   "string — anonymized user
identifier (optional)",
    "locale":   "string — BCP-47 language tag,
e.g. en-US"
  },
  "initStrategy": "string — COLD_START |
CONTEXT_SEEDED (optional, default COLD_START)",
  "callbackUrl":  "string — webhook URL for async
events (optional)"
}
```

Response (200 OK)

```
{
  "sessionId":     "string — UUID assigned to
this session",
  "startTimestamp":"string — ISO 8601 UTC
timestamp",
  "initStrategy":  "string — strategy used",
  "status":        "ACTIVE"
}
```

3.2.2 POST /hns/analyze

The primary endpoint. Analyzes a human utterance and computes updated HNS-36 coordinates for the session turn. Returns the structural snapshot and a natural-language feedback instruction.

Request Body

```
{
  "sessionId":    "string — required",
  "turnId":       "integer — required; must be
next sequential turn for session",
  "inputText":    "string — required; the human
utterance to analyze",
  "inputModality":"string — optional; TEXT
(default) | VOICE_TRANSCRIPT",
  "options": {
    "returnDirective": true,
    "validateCoherence": true,
    "learningRate": 0.6
```

```
  }
}
```

Response (200 OK)

```
{
  "snapshotId":     "string",
  "sessionId":      "string",
  "turnId":         "integer",
  "norm":           "float — RMS of all 36
coordinates",
  "priorDistance":  "float — Euclidean distance
from previous turn",
  "coherenceValid": "boolean",
  "coordinates": {

"l1":{"act":0.70,"sal":0.62,"coh":0.78,"con":0.44
,"int":0.66,"adp":0.53},

"l2":{"act":0.85,"sal":0.80,"coh":0.74,"con":0.32
,"int":0.75,"adp":0.63},

"l3":{"act":0.90,"sal":0.83,"coh":0.87,"con":0.41
,"int":0.81,"adp":0.69},

"l4":{"act":0.88,"sal":0.91,"coh":0.93,"con":0.34
,"int":0.79,"adp":0.74},

"l5":{"act":0.94,"sal":0.86,"coh":0.84,"con":0.26
,"int":0.77,"adp":0.79},

"l6":{"act":0.77,"sal":0.71,"coh":0.81,"con":0.31
,"int":0.74,"adp":0.84}
  },
  "structuralDirective": {
    "primaryLayer": 4,
    "activeLayers": [3, 4, 5],
    "intentionSummary": "User pursuing a specific
goal with high cognitive engagement.",
    "feedbackInstruction": "string — natural
language prompt injection text"
  },
  "processingLatencyMs": 142
}
```

3.2.3 POST /hns/evaluate

Computes the HNS-36 structural alignment score for a completed turn.

Request Body

```
{
  "sessionId":    "string — required",
  "turnId":       "integer — required",
  "snapshotId":   "string — snapshot from
/hns/analyze for this turn",
  "responseText": "string — the LLM response to
evaluate"
}
```

Response (200 OK)

```
{
  "evaluationId":   "string",
  "overallScore":   0.876,
  "structuralGrade":"AA",
  "layerScores": {
    "l1":0.820, "l2":0.855, "l3":0.890,
    "l4":0.932, "l5":0.878, "l6":0.881
  },
  "weakestDimension": "l1_con",
  "evaluationNotes": "Strong intentional
alignment. L1 constraint slightly low.",
  "evaEventRef": "urn:eva:event:eval-20260520-
abc"
}
```

3.2.4 POST /hns/feedback

Submits a Structural Feedback Signal (SFS) to correct a specific coordinate value. Call this when the AI system detects that the HNS-36 score is below threshold (< 0.6) and wishes to apply a correction.

Request Body

```
{
  "sessionId":          "string — required",
  "turnId":             "integer — required",
  "targetLayer":        "integer 1-6 — required",
  "targetAxis":         "string —
act|sal|coh|con|int|adp — null for full layer",
  "correctedValue":     "float 0.0-1.0 —
required",
  "correctionStrength":"float 0.0-1.0 — default
0.5",
  "rationale":          "string — recommended",
  "sourceType":         "AI_SYSTEM |
HUMAN_EVALUATOR"
```

```
}
```

3.2.5 POST /hns/structure

Commits a completed turn. Must be called after /hns/analyze (and optionally /hns/evaluate and /hns/feedback) to advance the session state.

Request Body

```
{
  "sessionId":    "string — required",
  "turnId":       "integer — required",
  "snapshotId":   "string — from /hns/analyze —
required",
  "responseText": "string — LLM response —
optional but recommended"
}
```

3.3 Typical Integration Flow

The following pseudocode shows the complete MVI turn cycle. This is the reference integration pattern for Day 1 implementation:

```
# ── Session start
──────────────────────────────────────────────────
──────
session = POST /hns/session/start
{ platformContext: {...} }
sessionId = session.sessionId

# ── Per-turn loop
──────────────────────────────────────────────────
──────
for turnId = 1, 2, 3, ...:

  # 1. User speaks
  userInput = receive_user_input()

  # 2. Structural analysis
  analysis = POST /hns/analyze {
    sessionId, turnId, inputText: userInput
  }

  # 3. Build LLM prompt with structural directive
  prompt = build_prompt(
    userInput,
```

```
    systemPrompt +
analysis.structuralDirective.feedbackInstruction
  )

  # 4. Generate response
  response = llm.generate(prompt)

  # 5. Evaluate structural alignment
  eval = POST /hns/evaluate {
    sessionId, turnId,
    snapshotId: analysis.snapshotId,
    responseText: response
  }

  # 6. Feedback if low score
  if eval.overallScore < 0.6:
    POST /hns/feedback { sessionId, turnId,
      targetLayer: weakest_layer(eval),
      correctedValue: suggest_correction(eval),
      sourceType: 'AI_SYSTEM' }

  # 7. Commit turn
  POST /hns/structure {
    sessionId, turnId, snapshotId:
analysis.snapshotId,
    responseText: response
  }

  # 8. Return response to user
  send_response(response)
```

3.4 Error Codes

Code	HTTP	Meaning	Resolution
HNS-4001	400	Missing required field.	Check request body against schema in Section 3.2.
HNS-4004	404	Session not found.	Verify sessionId; call POST /hns/session/start if session expired.
HNS-	409	Turn ID out of	Use the next sequential

Code	HTTP	Meaning	Resolution
4009		order.	turn ID. Retrieve current state via GET /hns/session/{id}.
HNS-4022	422	Structural analysis failed.	Check inputText is non-empty and within 32,768 character limit.
HNS-5003	503	HNS Core Engine unavailable.	Retry with exponential backoff starting at 1s. Check /hns/health endpoint.

4. Reference Use Cases

4.1 Overview

This section defines three PoC scenarios that are specifically selected to produce clear, measurable improvements within the 4-week PoC timeline. Each use case is designed to be demonstrable to both technical teams and business leadership: the before/after difference is observable in response quality, measurable via HNS-36 scores, and translatable into business terms.

All three use cases can be run concurrently or sequentially. For organizations with limited PoC resources, Use Case 1 (Intent Structuring) is the recommended starting point: it has the lowest implementation complexity, produces the most immediate visible improvement, and provides the clearest executive demonstration.

4.2 Use Case 1 — Intent Structuring

Problem Statement

LLMs process human language as a statistical sequence of tokens. They are highly capable at predicting linguistically appropriate responses, but they do not maintain an explicit model of the human agent's structural intent — the goals, cognitive frame, and constraints that underlie the utterance. As a result, LLM responses are frequently well-formed but structurally misaligned: they answer the literal question rather than serving the underlying purpose.

This misalignment is most visible in situations where the user's utterance is ambiguous, implicit, or indirect. A user who asks 'Is there a faster way to do this?' may be structurally asking for a performance optimization, a workflow redesign, a validation that their current approach is acceptable, or permission to abandon the task entirely. Without structural modeling, an LLM will guess the most statistically probable interpretation; with HNS, the correct interpretation is derived from the L4 (Intentional) and L3 (Cognitive) coordinates of the session.

HNS Effect

HNS Intent Structuring extracts the following structural properties from each user utterance and encodes them in the HNS-36 coordinate set:

- Role: The interactional role the user is occupying (e.g., professional problem-solver, learner, decision-maker, delegator).
- Intention: The immediate goal of the current utterance and its relationship to the session-level goal (L4 coordinates).
- Constraint: The implicit and explicit constraints that bound acceptable responses (L4 Constraint axis; L3 Coherence).
- Context: The situational frame that determines what counts as an appropriate response (L1 and L2 coordinates).

Scenario	User Utterance	Without HNS	With HNS (L4.act=0.92, L3.coh=0.88)
Ambiguous request	'Can you make this shorter?'	Reduces word count by 30%.	Identifies goal as 'professional brevity'; removes redundancy while preserving technical precision.
Implicit expertise signal	'I know the basics, let's skip that.'	Provides a brief summary of basics anyway.	L3 belief axis = 0.85 (high knowledge state); skips fundamentals; advances to expert-level content.
Goal shift mid-	'Actually, let's	Resets to generic	L4 goal axis shift detected;

Scenario	User Utterance	Without HNS	With HNS (L4.act=0.92, L3.coh=0.88)
session	approach this differently.'	approach.	maintains session context; restructures approach while preserving accumulated knowledge.
Constraint-driven request	'Keep it brief — I need to present this in 5 minutes.'	Provides comprehensive answer.	L4 constraint axis = 0.92 (high constraint); generates executive-summary-level response.

Implementation Steps

18. Enable Intent Structuring by passing the user utterance through POST /hns/analyze before constructing the LLM prompt.
19. Inject the returned structuralDirective.feedbackInstruction into the LLM system prompt as a structural context block.
20. Log the HNS-36 score from POST /hns/evaluate for each turn; track L4 (Intentional) and L3 (Cognitive) layer scores separately.
21. For PoC demonstration: select 20 representative interaction transcripts from your production system. Run them through HNS; compare response quality ratings before and after structural conditioning.

Success Metrics

- Mean HNS-36 L4 alignment score > 0.80 across all PoC turns.

- User-rated response relevance improvement of > 15% (measured via A/B comparison or annotator rating).
- Structural misalignment incidents (score < 0.6) reduced by > 30% compared to baseline.

4.3 Use Case 2 — Long-Horizon Consistency

Problem Statement

Extended AI dialogues — sessions of 30 or more turns — expose a fundamental limitation of context-window-based AI systems. As the conversation grows, the LLM's effective context window fills up, forcing either truncation of prior context (causing coherence loss) or expensive full-context processing (causing latency and cost increases). The result is a characteristic degradation in long-horizon AI interactions: responses become repetitive, fail to honor commitments made in earlier turns, and lose track of the user's evolving goals.

This problem is particularly acute in copilot applications, consultative dialogues, and any AI use case that requires building on prior turns. Users who have invested significant effort establishing context in early turns are especially frustrated when the AI 'forgets' this context in later turns.

HNS Effect

HNS structural persistence solves the long-horizon coherence problem by maintaining a compact (36-float) structural model of the interaction that persists across all turns, independent of the LLM's context window. The Structural Snapshot captures the session's structural trajectory in a form that is far more compact than the full conversation text, and far more structurally informative than a simple summary.

The key mechanism is the structural learning rate (alpha = 0.6 by default): coordinate values evolve smoothly across turns, with each new turn blending the new signal with the accumulated structural history. This means that even at turn 100, the HNS-36 coordinates faithfully represent the full structural arc of the interaction — not just the last few turns.

Measurement Protocol

To measure long-horizon consistency improvement, conduct the following experiment during the PoC:

22. Select or construct 10 interaction transcripts of 30-50 turns each, from a domain relevant to your use case.
23. Run each transcript twice: once without HNS (baseline) and once with HNS structural context injection.
24. At turns 10, 20, 30, and 50, have human annotators rate response consistency with the established session goals and context (5-point scale).
25. Compute the mean consistency rating at each turn checkpoint for baseline vs HNS conditions.
26. Expected result: HNS condition maintains consistency rating > 4.0/5.0 at all checkpoints; baseline condition shows degradation below 3.5/5.0 after turn 20-30.

Turn Checkpoint	Expected Baseline Consistency	Expected HNS Consistency	HNS Advantage
Turn 10	4.5 / 5.0	4.6 / 5.0	+0.1 (minimal — both systems performing well)
Turn 20	3.8 / 5.0	4.5 / 5.0	+0.7 (context compression beginning to affect baseline)
Turn 30	3.2 / 5.0	4.4 / 5.0	+1.2 (significant coherence gap opening)
Turn 50	2.7 / 5.0	4.3 / 5.0	+1.6 (baseline severely

Turn Checkpoint	Expected Baseline Consistency	Expected HNS Consistency	HNS Advantage
			degraded; HNS maintains coherence)

4.4 Use Case 3 — Safety and Governance (EVA)

Problem Statement

Enterprise AI deployments in regulated industries (financial services, healthcare, legal, government) face a fundamental auditability challenge: LLM reasoning is opaque. When an AI system produces a response — even a harmful or misleading one — there is no native mechanism to trace why that response was generated, what reasoning led to it, or whether the AI was behaving within defined safety boundaries. This opacity is not just a technical inconvenience; it is a regulatory liability under the EU AI Act, ISO/IEC 42001, and sector-specific AI governance frameworks.

HNS Effect

HNS+EVA transforms AI from an opaque system into an auditable one. At every turn, HNS records the structural state of the interaction — the human agent's goal coordinates, cognitive coherence, constraint profile, and expressive choices — in an append-only, cryptographically signed log. This log is the structural audit trail that enables:

- Post-hoc review of any AI response: auditors can retrieve the exact structural coordinates that were active when a given response was generated and assess whether the response was structurally appropriate.
- Real-time safety monitoring: the Goal Boundary Enforcement system triggers alerts when L4 (Intentional) coordinates cross predefined safety thresholds — before harmful content is generated.

- Regulatory submission: EVA logs are structured to satisfy Article 12 (Record-Keeping) requirements of the EU AI Act and ISO/IEC 42001 Section 9.4 (Internal Audit) requirements.
- Root-cause analysis: when an AI interaction goes wrong, the EVA log allows engineers to trace exactly when and how the structural state deviated from expected boundaries.

EVA Log Structure

Every EVA log event has the following structure:

```
{
  "evaEventId":    "urn:eva:event:coord-
update:20260520-abc123",
  "eventType":     "COORDINATE_UPDATE",
  "sessionId":     "sess-001",
  "turnId":        3,
  "timestamp":     "2026-05-20T10:30:00.123Z",
  "sequenceNumber": 7,
  "payload": { /* StructuralSnapshot or SFS or
EvaluationScore */ },
  "signature":     "sha256:a3f9bc2e...",
  "previousEventRef": "urn:eva:event:...:6"
}
```

Success Metrics for Safety PoC

- EVA logs produced for 100% of turns in all PoC sessions (zero gaps).
- At least one Goal Boundary Enforcement alert triggered and correctly escalated during PoC simulation.
- EVA log integrity verified via signature chain for all PoC sessions.
- EVA log structure validated against ISO/IEC 42001 Article 9.4 audit trail requirements.

5. Evaluation Protocol (HNS-36)

5.1 Overview

The HNS-36 Evaluation Protocol is the official method for measuring PoC success. It produces a quantitative, comparable, and auditable score that reflects how well the HNS-augmented AI system is structurally aligned with the human interlocutor at every turn. The protocol is designed to be runnable by any engineering team without specialized knowledge of HNS internals: the only inputs are the user utterance, the AI response, and the HNS-36 coordinate set computed by POST /hns/analyze.

5.2 Evaluation Metrics

The HNS-36 Evaluation Protocol produces scores on five dimensions, in addition to the overall HNS-36 score:

Metric	Definition	HNS Layer	Target (PoC Success)
Structural Accuracy	How closely the computed HNS-36 coordinates match the structural ground truth for the input (assessed against annotated reference data).	All layers	> 0.80 mean MAE on PoC test set
Role Consistency	How consistently the AI response honors the user's interactional role as encoded in the L3 Coherence and L4 Goal axes.	L3, L4	> 0.85 across all turns

Metric	Definition	HNS Layer	Target (PoC Success)
Intention Alignment	How well the AI response serves the user's actual goal (L4 Goal axis) vs. the surface content of the utterance.	L4	> 0.80 mean score
Constraint Handling	How well the AI response respects explicit and implicit constraints (L4 Constraint axis; L1 and L2 Constraint axes).	L1, L2, L4	> 0.75 compliance rate
Feedback Responsiveness	How effectively the structural coordinates improve after SFS events are applied; measured as the score delta between pre- and post-feedback turns.	All layers	> 0.10 mean score improvement per SFS

5.3 Scoring Methodology

5.3.1 Per-Turn Score

The per-turn HNS-36 score is computed as follows:

27. For each of the 36 coordinates C(l, a), compute the axis alignment value: A(l,a) = 1.0 - |C_human(l,a) - C_response(l,a)|, where C_human is the coordinate from

/hns/analyze and C_response is the coordinate inferred from the LLM response.

28. Compute the per-layer score: S(l) = mean(A(l,1), A(l,2), ..., A(l,6)) for each layer l in {1..6}.
29. Compute the overall score: S = mean(S(1), S(2), ..., S(6)).
30. Assign a structural grade: S >= 0.90 → AAA; S >= 0.75 → AA; S >= 0.60 → A; S < 0.60 → Below A.

5.3.2 Session Score

The session HNS-36 score is the mean of all per-turn scores for the session. Additionally, the session evaluation reports:

- Structural Grade Distribution: percentage of turns at each grade level.
- Weakest Layer: the layer with the lowest mean alignment score across all turns.
- Structural Improvement Trend: the linear regression slope of the per-turn score over the session, indicating whether structural alignment is improving, stable, or degrading.
- Feedback Effectiveness: the mean score improvement for turns where SFS events were applied.

5.4 PoC Success Thresholds

The following thresholds define PoC success levels. Teams should agree on the target success level with leadership before beginning the PoC.

Success Level	Overall HNS-36 Score	Grade Distribution	Business Interpretation
Baseline (no HNS)	< 0.65 (expected)	< 50% at A or above	Current state without HNS structural conditioning.
Level A	≥ 0.70	≥ 70% of	Basic structural

Success Level	Overall HNS-36 Score	Grade Distribution	Business Interpretation
(MVI Success)		turns at A or above	alignment achieved. HNS is working correctly; refinement needed.
Level AA (PoC Target)	≥ 0.80	≥ 80% of turns at AA or above	Strong structural alignment. Production-ready for non-safety-critical applications.
Level AAA (Excellence)	≥ 0.90	≥ 90% of turns at AAA	Exceptional structural alignment. Production-ready for safety-critical applications; suitable for regulatory submission.

5.5 PoC Evaluation Report Template

At the end of Week 4, the PoC team should produce an evaluation report containing the following sections. A Word template for this report is available in the 03_NSW GitHub repository.

- Executive Summary: One-page summary of PoC outcomes, HNS-36 scores achieved, and recommendation.
- Test Data Description: Number of sessions, turns, and domains tested; data collection methodology.
- Baseline Scores: HNS-36 scores from the pre-HNS baseline (random or rule-based coordinate assignment).

- PoC Scores: HNS-36 scores from the HNS-augmented system; breakdown by use case, layer, and session.
- Before/After Comparison: Side-by-side comparison of selected interaction transcripts with and without HNS.
- Metric Results: Results for all five evaluation metrics (Section 5.2) with comparison to PoC success thresholds.
- EVA Log Sample: Sample EVA log events demonstrating audit trail completeness and format.
- Issues and Resolutions: Any structural misalignment patterns observed and how they were addressed.
- Recommendation: Proceed to full HNS deployment / Extend PoC / Discontinue, with justification.

6. Logging and Auditability (HSF / EVA)

6.1 Overview

Comprehensive, structured logging is not an optional feature of HNS — it is a core architectural commitment. Every structural event in an HNS session produces a log entry that is stored in an append-only format, enabling complete reconstruction of the session's structural history at any later time. For regulated industry deployments, this log is the evidence base for regulatory compliance; for engineering teams, it is the primary debugging and optimization tool; for business stakeholders, it is the transparency layer that makes AI behavior explainable.

The HNS logging system has two components: HSF (HNS Structural Feedback) logs, which capture structural feedback events within the processing pipeline, and EVA (External Verification Architecture) logs, which capture all structural events for external audit. In the MVI, these are implemented as a single integrated log store (EVA-Lite) that satisfies the requirements of both.

6.2 Required Logs

The following log types MUST be produced by all MVI implementations:

Log Type	Trigger	Mandatory Fields	Purpose
SESSION_START	POST /hns/session/start called	sessionId, timestamp, initStrategy, platformContext	Establishes session boundary; enables session-level audit

Log Type	Trigger	Mandatory Fields	Purpose
			scoping.
COORDINATE_UPDATE	POST /hns/analyze completes successfully	sessionId, turnId, snapshotId, norm, priorDistance, coherenceValid	Primary structural event log; the core of the audit trail.
SFS_RECEIVED	POST /hns/feedback called	sessionId, turnId, sfsId, targetLayer, targetAxis, currentValue, correctedValue, sourceType	Records every corrective signal; enables feedback effectiveness analysis.
SFS_APPLIED	SFS successfully applied to coordinates	sfsId, priorValue, correctedValue, deltaApplied	Confirms correction was applied; records actual delta.
EVALUATION_SCORE	POST /hns/evaluate completes	evaluationId, sessionId, turnId, overallScore, structuralGrade, layerScores	Records per-turn alignment scores; primary PoC evaluation data.
SESSION_END	POST	sessionId,	Closes

Log Type	Trigger	Mandatory Fields	Purpose
	/hns/session/end called	endTimestamp, totalTurns, sessionNorm, sessionGrade	session boundary; records session-level summary.

6.3 EVA-Lite Log Format

The EVA-Lite log format used in MVI implementations is a simplified version of the full EVA log format. EVA-Lite events are written as newline-delimited JSON (NDJSON) to a log file or append-only database table. Each event is a self-contained JSON object:

```
{
  "evaEventId":      "string — unique event
identifier (UUID recommended)",
  "eventType":       "string — one of the log
types defined in Section 6.2",
  "sessionId":       "string",
  "turnId":          "integer or null (null for
SESSION_START/END)",
  "timestamp":       "string — ISO 8601 UTC with
millisecond precision",
  "sequenceNumber":  "integer — monotonically
increasing per-session counter",
  "payload":         "object — event-specific
data (see Section 6.2)",
  "checksum":        "string — SHA-256 of
(sequenceNumber + payload JSON)"
}
```

> **Important:** The checksum field is REQUIRED for regulatory submissions. For PoC deployments not targeting regulatory compliance, the checksum may be omitted but is strongly recommended as a data integrity safeguard.

6.4 Logging for Transparency

Beyond regulatory compliance, HNS structural logs enable a form of AI transparency that is qualitatively different from post-hoc explanation methods (such as attention visualization or saliency maps). HNS logs are prospective and structural: they record what the AI system understood about the human agent's structural state before generating a response, making it possible to answer the question 'Why did the AI respond that way?' with a precise, quantitative structural answer.

For PoC demonstration purposes, the following query should be implementable against the EVA-Lite log store within the PoC period: 'For session {sessionId}, retrieve all turns where the L4 (Intentional) Goal activation exceeded 0.85, and show the corresponding AI response and HNS-36 score.' This query demonstrates to business stakeholders that the AI's reasoning is not only transparent but queryable.

6.5 Logging for Debugging

During PoC development, the EVA-Lite log is the primary debugging tool for structural misalignment issues. The following debugging patterns are recommended:

Pattern 1: Score Degradation Analysis

If the per-turn HNS-36 score is declining across a session, retrieve the COORDINATE_UPDATE log entries and plot the norm and priorDistance values over turns. Increasing priorDistance indicates structural drift; high drift combined with low scores suggests that the coordinate update algorithm needs tuning (increase or decrease alpha).

Pattern 2: Feedback Effectiveness Analysis

If SFS_APPLIED events are not improving the subsequent turn's HNS-36 score, check the correctionStrength and deltaApplied fields. Low deltas may indicate that the correction is being over-damped by the alpha blending; consider increasing correctionStrength for high-confidence corrections.

Pattern 3: Coherence Violation Tracking

If COORDINATE_UPDATE events frequently have

coherenceValid=false, inspect which coherence constraints are being violated. Frequent CC-01 violations (Layer Monotonicity) suggest that the signal extraction for lower layers is producing implausibly low activation values; review the input preprocessing pipeline.

7. Deployment Guide

7.1 Overview

This section provides step-by-step deployment instructions for three infrastructure configurations: Local (for development and testing), Cloud (for scalable production), and Hybrid (for regulated industries requiring data residency). All three configurations produce a fully functional MVI system; the choice of configuration affects infrastructure cost, latency, scalability, and data governance, not structural functionality.

For PoC purposes, the Local deployment is the fastest path to a running system and is the recommended starting point for all teams. Cloud and Hybrid deployments can be pursued after the PoC validates the structural approach.

7.2 Recommended Minimal Stack

All three deployment models share the same application-layer stack:

Component	Recommended Technology	PoC Alternative	Notes
HNS Core Engine	Python 3.10+ service (FastAPI)	Any HTTP server returning valid HNS-36 JSON	The coordinate computation algorithm is implementer-defined for PoC.
HSF-Lite	Integrated with HNS Core Engine (same process)	Standalone Python script	For MVI, HSF-Lite can be a thin wrapper on the Core Engine.

Component	Recommended Technology	PoC Alternative	Notes
Structural Memory Store	PostgreSQL 14+ (recommended) or SQLite (PoC only)	Any key-value store with JSON support	Must support concurrent reads; append semantics for EVA events.
EVA-Lite Logger	Append-only table in PostgreSQL, or NDJSON file	NDJSON file on local filesystem	File-based EVA-Lite is acceptable for PoC; not recommended for production.
API Gateway	FastAPI (Python) or Express (Node.js)	Flask or any HTTP framework	Must support JSON body parsing and CORS for browser-based integrations.
LLM Integration	HTTP client calling existing LLM API	Any LLM accessible via HTTP	HNS is LLM-agnostic; connect to your existing LLM service.

7.3 Local Deployment (Day 1 Setup)

The following instructions set up a running MVI on a single development machine in approximately 2 hours.

Step 1: Environment Setup

```
# Prerequisites: Python 3.10+, pip, git
git clone https://github.com/satoru-hara/03_NSW.git
cd 03_NSW
python3 -m venv hns-env
source hns-env/bin/activate        # Linux/macOS
```

```
hns-env\Scripts\activate          # Windows
pip install fastapi uvicorn sqlite3-api pydantic
```

Step 2: Initialize the Structural Memory Store

```
# SQLite for local PoC
python3 scripts/init_db.py --backend sqlite --
path ./hns_poc.db
# Verify:
python3 scripts/check_db.py --path ./hns_poc.db
```

Step 3: Start the HNS Core Engine

```
uvicorn hns.main:app --host 0.0.0.0 --port 8080 -
-reload
# Verify:
curl http://localhost:8080/hns/health
# Expected: { "status": "healthy", "components":
{...} }
```

Step 4: Run the Smoke Test

```
python3 scripts/smoke_test.py --base-url
http://localhost:8080
# Expected output:
#   [PASS] POST /hns/session/start
#   [PASS] POST /hns/analyze — valid HNS-36
coordinates returned
#   [PASS] POST /hns/evaluate — score in [0.0,
1.0]
#   [PASS] POST /hns/feedback — correction applied
#   [PASS] POST /hns/structure — turn committed
#   [PASS] POST /hns/session/end
#   [PASS] EVA log: 6 events written, checksum
chain valid
#  All smoke tests PASSED. MVI is operational.
```

Step 5: Connect Your LLM

Update the LLM configuration file (config/llm.yaml) with your LLM provider credentials and endpoint. The HNS Core Engine calls the LLM for response-side coordinate computation in the evaluation step; it does NOT call the LLM for input-side coordinate computation in the analysis step.

> **Tip:** For the first 24 hours of the PoC, you can substitute a rule-based coordinate extractor for the HNS Core Engine algorithm. The key is to get the API layer working and producing valid HNS-36 JSON. Structural accuracy can be improved iteratively after the infrastructure is validated.

7.4 Cloud Deployment

For cloud deployment, the MVI stack maps directly to standard cloud-native components. The following mapping applies to all major cloud platforms (AWS, Azure, GCP):

MVI Component	AWS	Azure	GCP
HNS Core Engine	ECS (Fargate) or Lambda	Container Apps or Functions	Cloud Run or GKE
Structural Memory Store	RDS PostgreSQL	Azure Database for PostgreSQL	Cloud SQL (PostgreSQL)
EVA-Lite Logger	CloudWatch Logs + S3	Azure Monitor + Blob Storage	Cloud Logging + GCS
API Gateway	API Gateway + ALB	API Management	Cloud Endpoints + LB
LLM Integration	Bedrock or external API	Azure OpenAI or external API	Vertex AI or external

7.5 Hybrid Deployment

Hybrid deployment is designed for organizations with data residency requirements — where user interaction data (utterances and structural coordinates) must remain on-premises or within a

specific geographic boundary, while audit logs may be hosted in a cloud-based verification service accessible to regulators.

In hybrid deployment: the HNS Core Engine, HSF-Lite, and Structural Memory Store are deployed on-premises or in a private cloud. The EVA-Lite logger writes to a local buffer. A scheduled export job (running every 1-15 minutes, configurable) ships the EVA log events to a cloud-hosted EVA verification endpoint. User input text is never included in the exported events; only structural coordinates and metadata are exported.

> **Important:** In hybrid deployments, ensure that the EVA export job has failover handling: if the cloud EVA endpoint is unavailable, events must be queued locally and retried with exponential backoff. A gap in the EVA event sequence will be flagged as a potential integrity violation by regulatory auditors.

8. 4-Week PoC Timeline

8.1 Timeline Overview

The 4-week PoC timeline is structured around four milestone weeks, each building on the previous. The timeline is designed to be followed by a 2-person team (one backend engineer and one AI/research specialist) working at approximately 50% allocation. Teams with more resources can accelerate; teams with less should prioritize Weeks 1 and 2 and treat Weeks 3 and 4 as extensions.

Week	Focus	Primary Deliverable	Success Gate
Week 1	Infrastructure & Integration	Running MVI: all APIs responding, first HNS-36 score computed	Smoke test PASSED on Day 5
Week 2	Use Case 1: Intent Structuring	Intent Structuring PoC with before/after comparison on 20 test cases	Mean L4 alignment score ≥ 0.75
Week 3	Use Cases 2 & 3: Consistency + Safety	Long-horizon test results (30-turn sessions) + EVA audit trail demo	Mean session score ≥ 0.78; EVA log complete
Week 4	Evaluation & Report	Full PoC evaluation report + executive presentation	Overall HNS-36 score ≥ 0.80 (Level AA target)

8.2 Week 1 — Infrastructure and Integration

Objectives

- Deploy the MVI stack (HNS Core Engine, HSF-Lite, EVA-Lite, Structural Memory Store).
- Implement all mandatory API endpoints and pass the smoke test.
- Connect HNS to the target LLM; verify end-to-end turn cycle.
- Compute the first HNS-36 baseline score on real interaction data.

Day-by-Day Plan

Day	Task	Owner	Hours
Day 1	Environment setup; install dependencies; initialize DB; start HNS Core Engine.	Engineer	4h
Day 2	Implement POST /hns/analyze with basic coordinate extraction; run unit tests.	Engineer	6h
Day 3	Implement POST /hns/structure, /hns/feedback, /hns/evaluate; implement EVA-Lite.	Engineer	6h
Day 4	Implement session management endpoints; run full smoke test; fix failing tests.	Engineer	6h
Day 5	Connect LLM; run first end-to-end turn cycle; compute first HNS-36 score.	Engineer + AI Specialist	4h

Week 1 Deliverables

- Smoke test report (pass/fail for all 7 MVI tests).
- Screenshot or log of first successful HNS-36 score computation.

- EVA-Lite log sample showing 6 event types for a test session.

8.3 Week 2 — Use Case 1: Intent Structuring

Objectives

- Assemble 20 representative test cases from production interaction data (or construct synthetic ones).
- Run all 20 test cases through both baseline (no HNS) and HNS-augmented conditions.
- Compute per-turn and per-test-case HNS-36 scores for both conditions.
- Produce before/after comparison for 5 selected test cases suitable for executive presentation.

Day-by-Day Plan

Day	Task	Owner	Hours
Day 6	Collect and anonymize 20 test interaction transcripts.	AI Specialist	4h
Day 7	Build baseline evaluation pipeline (no HNS context injection); run all 20 cases.	Engineer + AI Specialist	5h
Day 8	Enable HNS structural directive injection; run all 20 cases with HNS.	Engineer + AI Specialist	5h
Day 9	Score all 40 transcripts (baseline + HNS) using HNS-36 Evaluator; compute metrics.	AI Specialist	4h
Day 10	Select 5 best before/after pairs; write Week 2 summary report.	AI Specialist	3h

8.4 Week 3 — Use Cases 2 and 3

Objectives

- Run long-horizon consistency test (Use Case 2): 10 sessions × 30-50 turns each.
- Measure consistency scores at turns 10, 20, 30, 50 for baseline and HNS conditions.
- Run safety/governance PoC (Use Case 3): verify EVA audit trail completeness and integrity.
- Simulate one Goal Boundary Enforcement alert scenario; document escalation behavior.

Day-by-Day Plan

Day	Task	Owner	Hours
Day 11	Prepare 10 long-horizon test sessions (30-50 turn transcripts or synthetic).	AI Specialist	5h
Day 12	Run long-horizon baseline condition; score at each checkpoint turn.	Engineer + AI Specialist	6h
Day 13	Run long-horizon HNS condition; score at each checkpoint turn; compare to baseline.	Engineer + AI Specialist	6h
Day 14	Configure Goal Boundary thresholds; run Safety PoC scenario; verify EVA alerts.	Engineer	5h
Day 15	Verify EVA log integrity (checksum chain); prepare Week 3 data summary.	AI Specialist	3h

8.5 Week 4 — Evaluation and Report

Objectives

- Aggregate all PoC evaluation data: per-turn scores, session scores, metric results.
- Write the full PoC Evaluation Report (using the template in Section 5.5).
- Prepare the executive presentation: 10-15 slides covering outcomes and recommendation.
- Conduct the Week 4 leadership review meeting.

Day-by-Day Plan

Day	Task	Owner	Hours
Day 16	Aggregate all HNS-36 scores; compute final metric values for all 5 evaluation dimensions.	AI Specialist	5h
Day 17	Write Use Case results sections of the evaluation report; produce comparison charts.	AI Specialist	5h
Day 18	Write Executive Summary and Recommendation sections of report; internal review.	AI Specialist + PM	4h
Day 19	Prepare executive presentation slides; rehearse.	PM + AI Specialist	4h
Day 20	Leadership review meeting; present results; decide on next steps.	Full team	2h

8.6 PoC Outcomes Decision Framework

At the end of Week 4, the PoC leadership review should result in one of three decisions:

Decision	Criteria	Recommended Next Steps
Proceed to Full Deployment	Overall HNS-36 score ≥ 0.80; at least 2 of 3 use cases meet success thresholds; engineering team confident in implementation.	Engage HNS commercial licensing (see Licensing section of HNS-IS v1.0); scope full production deployment; assign dedicated HNS engineering resources.
Extend PoC (4 additional weeks)	Score ≥ 0.70 but < 0.80; some use cases successful but others need refinement; specific technical issues identified and addressable.	Address identified gaps; rerun failing scenarios; target Level AA in extended period.
Pause and Reassess	Score < 0.70; fundamental integration issues unresolved; use cases not demonstrating expected improvement.	Review HNS Core Engine implementation quality; consult HNS technical documentation; consider engaging implementation support.

Appendix A: Glossary

Key terms used in this PoC Package:

Term	Abbreviation	Definition
Human Natural Structure	HNS	A Structural Operating System that models the six-layer causal architecture of human cognition and meaning generation.
HNS-36 Coordinate System	HNS-36	A 36-dimensional coordinate system (6 layers × 6 axes) encoding the structural state of a human agent at any interaction turn.
Human Natural Layers	HNL	The six causal layers: Physical (L1), Perceptual (L2), Cognitive (L3), Intentional (L4), Expressive (L5), Reflexive (L6).
Structural Snapshot	—	An immutable record of the HNS-36 coordinate set for a specific turn, committed at turn closure.
Feedback Instruction	—	A natural-language prompt injection derived from the current HNS-36 coordinates; guides LLM response generation.
Structural Feedback Signal	SFS	A structured corrective message identifying a coordinate misalignment and specifying a target

Term	Abbreviation	Definition
		correction value.
HNS Structural Feedback Engine	HSF	The component that processes, applies, and logs SFS events.
HSF-Lite	—	The simplified version of HSF used in MVI deployments; processes externally submitted SFS events only.
External Verification Architecture	EVA	The independent, append-only logging system for structural events; enables external audit and regulatory compliance.
EVA-Lite	—	The simplified EVA implementation used in MVI deployments; produces NDJSON logs with checksum chain.
Minimal Viable Implementation	MVI	The smallest functional HNS configuration: HNS Core Engine + HSF-Lite + HNS-36 Evaluator.
Structural Grade	—	A letter grade (A/AA/AAA/Below A) assigned to a turn or session based on the HNS-36 alignment score.
Goal Boundary Enforcement	GBE	A safety mechanism that detects when L4 Intentional coordinates cross predefined safety thresholds.
Structural	alpha	The blending factor

Term	Abbreviation	Definition
Learning Rate		controlling how rapidly coordinates respond to new inputs; default 0.6.

Appendix B: MVI Smoke Test Reference

B.1 Smoke Test Overview

The MVI Smoke Test is a 5-minute automated validation that verifies all mandatory MVI requirements are met. Run it on Day 5 of Week 1 to confirm your implementation is ready for PoC scenario testing. The test script is available at scripts/smoke_test.py in the 03_NSW repository.

B.2 Smoke Test Sequence

Test ID	Test Name	Validates	Pass Criteria
ST-01	Session Start	POST /hns/session/start	Returns sessionId; status = ACTIVE.
ST-02	Analyze Turn 1	POST /hns/analyze	Returns all 6 layers, all 6 axes; norm in (0,1]; coherenceValid = true or false.
ST-03	Analyze Turn 2	POST /hns/analyze (2nd)	priorDistance > 0 (coordinates evolved); turnId = 2 accepted.
ST-04	Evaluate Turn 2	POST /hns/evaluate	overallScore in [0.0, 1.0]; structuralGrade assigned; evaEventRef non-empty.
ST-05	Feedback	POST /hns/feedback	Returns sfsId; applied = true; correctedValue matches request.

Test ID	Test Name	Validates	Pass Criteria
ST-06	Commit Turn 2	POST /hns/structure	committed = true; sessionTurnCount = 2.
ST-07	Session End	POST /hns/session/end	Returns totalTurns = 2; sessionNorm computed; EVA SESSION_END event logged.
ST-08	EVA Integrity	EVA-Lite log validation	6 events in log; sequenceNumbers 1-6 contiguous; all checksums valid.

B.3 Quick Troubleshooting

Symptom	Likely Cause	Resolution
ST-02 fails: coordinates contain nulls	HNS Core Engine not extracting signals for all layers	Verify that the coordinate computation function returns values for l1 through l6.
ST-03 fails: priorDistance = 0	Structural Memory Store not persisting Turn 1 snapshot	Check database write permissions; verify that /hns/analyze is calling the commit step.
ST-04 fails: score = null	LLM not connected; response-side coordinates cannot be computed	Verify LLM API credentials; check config/llm.yaml; test LLM connection independently.

Symptom	Likely Cause	Resolution
ST-08 fails: checksum invalid	EVA log writer computing checksum incorrectly	Verify SHA-256 is computed over (sequenceNumber + payload JSON); check JSON serialization consistency.

Appendix C: Reference Architecture

C.1 MVI Component Diagram

The MVI architecture (described textually; diagrams available in 03_NSW/Architecture-Diagrams/):

- [Human Interlocutor] → (utterance) → [AI System]
- [AI System] → POST /hns/analyze → [HNS Core Engine]
- [HNS Core Engine] ↔ [Structural Memory Store] (read prior state; write snapshot)
- [HNS Core Engine] → (coordinates + directive) → [AI System]
- [AI System] → (prompt + directive) → [LLM]
- [LLM] → (response) → [AI System]
- [AI System] → POST /hns/evaluate → [HNS-36 Evaluator]
- [HNS-36 Evaluator] → (score < 0.6?) → POST /hns/feedback → [HSF-Lite]
- [HSF-Lite] → (correction applied) → [Structural Memory Store]
- [AI System] → POST /hns/structure → [Structural Memory Store]
- [HSF-Lite] + [HNS Core Engine] → (all events) → [EVA-Lite Logger]
- [EVA-Lite Logger] → (NDJSON events) → [EVA Log Store]

C.2 Data Flow per Turn

Step	Actor	Action	Artifact Produced
1	Human	Provides utterance	Raw text input
2	AI System	Calls POST /hns/analyze	Analysis request

Step	Actor	Action	Artifact Produced
3	HNS Core Engine	Computes HNS-36 coordinates	StructuralSnapshot + Feedback Instruction
4	AI System	Builds LLM prompt with directive	Enriched prompt
5	LLM	Generates response	AI response text
6	AI System	Calls POST /hns/evaluate	Evaluation request
7	HNS-36 Evaluator	Computes alignment score	HNS-36 score + grade
8	AI System	Calls POST /hns/feedback if needed	SFS event (if score < 0.6)
9	AI System	Calls POST /hns/structure	Committed turn record
10	EVA-Lite	Logs all events	Append-only audit record

Appendix D: PoC Readiness Checklist

D.1 Pre-PoC Checklist (Complete Before Day 1)

- PoC scope and target use cases agreed with leadership (Section 1, Section 4).
- Engineering team allocated (minimum: 1 backend engineer at 50% allocation for 4 weeks).
- Development infrastructure provisioned (local or cloud, per Section 7).
- LLM API access configured and tested independently.
- Test interaction transcripts collected and anonymized (minimum 20 for Use Case 1).
- PoC success thresholds agreed with leadership (Section 5.4).
- Week 4 leadership review meeting scheduled.

D.2 Week 1 Completion Checklist

- MVI smoke test PASSED (all 8 tests, Appendix B).
- First HNS-36 score computed and logged.
- EVA-Lite log producing valid NDJSON with checksum chain.
- LLM connected; structural directive injected into test prompt.

D.3 Week 4 Completion Checklist

- HNS-36 scores computed for all PoC scenarios (≥ 100 turns total).
- All 5 evaluation metrics computed (Section 5.2).
- Before/After comparison prepared for at least 2 use cases.
- EVA audit trail integrity verified for all PoC sessions.
- PoC Evaluation Report written and reviewed.

- Executive presentation prepared.
- Leadership review conducted; next-steps decision made.

Appendix E: Change Log

Version 1.0 (May 2026) — Initial Release

This is the first public release of the HNS PoC Package. It is companion to HNS Implementation Specification v1.0 (Book 16) and shares the same API surface and data model. The PoC Package is intentionally prescriptive: it trades configuration flexibility for speed of first deployment, targeting the 24-hour-to-first-score goal.

Feedback on the PoC Package — including implementation experiences, timing observations, and use case suggestions — is welcomed via the 03_NSW GitHub repository discussion forum. Community-contributed PoC experiences will be incorporated into future versions.

— End of HNS PoC Package v1.0 —